

UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Projet PF22

RAPPORT DE PROJET

**Réduction des Coûts d'Apprentissage
par Approche Multi-Fidélité**

Ce projet explore la possibilité de réduire drastiquement les coûts de constitution d'un jeu de données en combinant une faible proportion d'images **Haute Fidélité** avec une grande majorité d'images **Basse Fidélité** acquises à moindre coût.

Auteur : Ivann Vasic

Période : Mars – Juin 2026

Cadre : Projet PF22 — UTBM

Résumé

L'entraînement de modèles d'apprentissage profond performants nécessite traditionnellement de vastes ensembles de données de haute qualité, dont l'acquisition et l'annotation s'avèrent souvent coûteuses en temps et en ressources. Ce projet explore une approche alternative : **l'apprentissage multi-fidélité**.

L'objectif est de démontrer qu'il est possible de réduire drastiquement les coûts de constitution d'un jeu de données en combinant une faible proportion d'images de **Haute Fidélité (HF)** avec une grande majorité d'images dégradées acquises à moindre coût (**Basse Fidélité - BF**).

Mots-clés : Apprentissage profond · Multi-fidélité · CNN · ResNet-18 · Dégradation d'image · Optimisation des coûts · Domain Shift · Transfer learning · Curriculum learning · EWC · Optuna · Animals-10 · Imagewoof · Intel

Table des matières

1	Notions Préliminaires	5
1.1	Les réseaux de neurones artificiels	5
1.2	Paramètres et poids du réseau	5
1.3	La boucle d'entraînement	6
1.3.1	La fonction de perte	6
1.3.2	La descente de gradient et l'optimiseur	6
1.4	Hyperparamètres clés	7
1.5	Sous-apprentissage, sur-apprentissage et généralisation	7
1.6	Les différentes dimensions du coût en apprentissage profond	8
2	Fondation et construction de l'environnement Multi-Fidélité	9
2.1	Environnement de travail et architecture hybride	9
2.2	Sélection du jeu de données	10
2.3	Découpage et répartition des données	12
2.4	Pipeline de dégradation de la qualité	12
2.5	Définition de la métrique de coût	13
2.6	Sélection et justification de l'architecture neuronale	14
2.6.1	Mécanique interne de ResNet-18	15
2.6.2	Justification des hyperparamètres et de la fonction de coût	15
2.6.3	Prétraitement et optimisations matérielles	16
3	Construction des Modèles de Référence	17
3.1	Stratégie d'évaluation croisée face au décalage de domaine	17
3.2	Définition des modèles de référence	17
3.3	Stratégie d'implémentation logicielle	18
3.4	Analyse des résultats	18
4	Stratégies Multi-Fidélité	23

4.1	Stratégie 1 – Transfer Learning Séquentiel (BF $\rightarrow$ HF)	23
4.1.1	Protocole	23
4.1.2	Résultats quantitatifs obtenus	24
4.1.3	Comparaison avec les baselines du Chapitre 3	26
4.1.4	Analyse : ce qui a marché, ce qui a moins marché, et enseignements	27
4.1.5	Améliorations possibles de la méthode (Stratégie 1)	27
4.2	Stratégie 2 – Co-training par Batch avec Pondération (Reweightings)	28
4.2.1	Protocole	28
4.2.2	Résultats quantitatifs obtenus	29
4.2.3	Comparaison avec les baselines et la Stratégie 1	31
4.2.4	Analyse : ce qui a marché, ce qui a moins marché, et enseignements	32
4.2.5	Améliorations possibles de la méthode (Stratégie 2)	33
4.3	Stratégie 3 – Curriculum Learning progressif	33
4.3.1	Protocole	34
4.3.2	Résultats quantitatifs obtenus	34
4.3.3	Comparaison avec les baselines et les stratégies précédentes	35
4.3.4	Analyse : ce qui a marché, ce qui a moins marché, et enseignements	35
4.3.5	Améliorations possibles de la méthode (Stratégie 3)	35
4.4	Stratégie 4 – Elastic Weight Consolidation (EWC)	36
4.4.1	Protocole	36
4.4.2	Résultats quantitatifs obtenus	37
4.4.3	Comparaison globale des stratégies	37
4.4.4	Analyse : ce qui a marché, ce qui a moins marché, et enseignements	39
4.4.5	Améliorations possibles de la méthode (Stratégie 4)	39

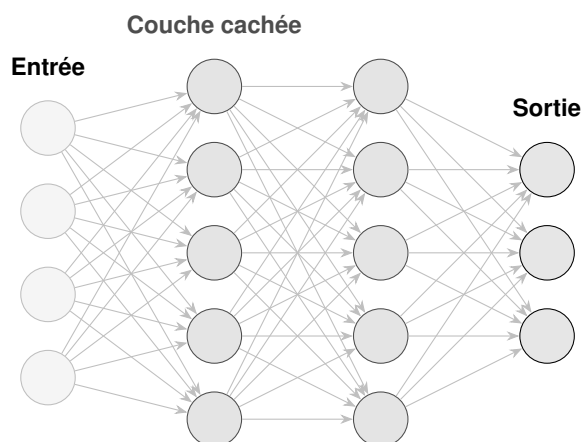
CHAPITRE 1

Notions Préliminaires

Ce chapitre présente les concepts fondamentaux nécessaires à la compréhension du projet. Le lecteur familier avec l'apprentissage profond peut passer directement au Chapitre 2.

1.1 Les réseaux de neurones artificiels

Un réseau de neurones artificiel est un système de calcul inspiré du cerveau biologique. Il est constitué d'unités élémentaires, les **neurones**, organisées en **couches** successives. Chaque neurone reçoit des signaux numériques en entrée, les combine linéairement, puis applique une **fonction d'activation** non-linéaire avant de transmettre le résultat à la couche suivante.



L'empilement de plusieurs couches cachées constitue un réseau dit **profond** (*deep network*). La profondeur permet au réseau d'apprendre des représentations de plus en plus abstraites : les premières couches détectent des motifs simples (contours, textures), tandis que les couches profondes reconnaissent des concepts complexes (forme d'un animal, expression d'un visage).

1.2 Paramètres et poids du réseau

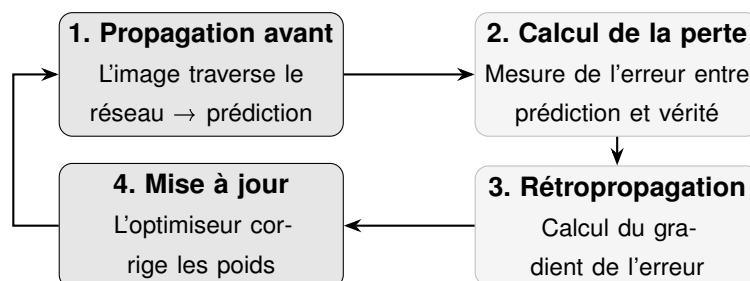
Les connexions entre neurones sont régies par des valeurs numériques appelées **poids** (w) et **biais** (b). Ce sont les **paramètres apprenables** du réseau : ils sont initialisés aléatoirement, puis ajustés itérativement au cours de l'entraînement.

Paramètre	Rôle	Interprétation
Poids w_{ij}	Pondère la connexion entre le neurone i et j	Fort poids $\Rightarrow$ le signal du neurone i influence fortement j
Biais b_j	Décale le seuil d'activation du neurone j	Permet au neurone de s'activer même en absence de signal fort

Un réseau comme ResNet-18 possède environ **11 millions de paramètres**. La qualité de l'apprentissage dépend de la capacité à trouver les valeurs optimales de l'ensemble de ces paramètres.

1.3 La boucle d'entraînement

L'entraînement d'un réseau de neurones est un processus itératif qui suit quatre étapes fondamentales, répétées pour chaque lot (*batch*) d'images :



Une **époque** (*epoch*) correspond au passage complet de l'intégralité du jeu de données d'entraînement à travers cette boucle. L'entraînement se déroule sur plusieurs dizaines d'époques jusqu'à convergence.

1.3.1 La fonction de perte

La **fonction de perte** (*loss function*) quantifie l'écart entre la prédiction du réseau et la vérité terrain. Plus la perte est faible, plus le modèle est précis. Pour la classification multi-classes, la **Entropie Croisée** (*Cross-Entropy Loss*) est le standard :

$$\mathcal{L} = - \sum_{c=1}^C y_c \cdot \log(\hat{p}_c)$$

où C est le nombre de classes, y_c la vraie étiquette (0 ou 1), et $\hat{p}_c$ la probabilité prédite par le réseau pour la classe c .

1.3.2 La descente de gradient et l'optimiseur

La **rétropropagation** calcule, pour chaque paramètre du réseau, le gradient de la perte : une mesure indiquant dans quelle direction et de quelle amplitude modifier ce paramètre pour réduire l'erreur. L'**optimiseur** utilise ensuite ces gradients pour mettre à jour les poids :

$$w \leftarrow w - \eta \cdot \nabla_w \mathcal{L}$$

où η est le **taux d'apprentissage** (*learning rate*), un hyperparamètre critique qui contrôle la taille du pas de correction. Un η trop grand risque de faire diverger l'apprentissage ; trop petit, la convergence devient extrêmement lente.

1.4 Hyperparamètres clés

Contrairement aux poids, les **hyperparamètres** ne sont pas appris automatiquement : ils doivent être fixés manuellement avant l'entraînement et conditionnent fortement la qualité du résultat final.

Hyperparamètre	Définition	Impact
Taux d'apprentissage η	Amplitude du pas de correction	Trop grand : divergence. Trop petit : apprentissage lent ou bloqué.
Taille de lot (<i>batch size</i>)	Nombre d'images traitées simultanément	Influence la stabilité du gradient et l'utilisation mémoire du GPU.
Nombre d'époques	Nombre de passages complets sur les données	Trop faible : sous-apprentissage. Trop élevé : mémorisation (<i>overfitting</i>).
Architecture	Nombre et type de couches	Détermine la capacité d'abstraction et la vitesse d'inférence.

1.5 Sous-apprentissage, sur-apprentissage et généralisation

L'objectif de l'entraînement n'est pas de mémoriser parfaitement les données d'entraînement, mais de **généraliser** : être capable de classer correctement des images jamais vues. Deux pathologies opposées peuvent survenir :

Phénomène	Cause	Symptôme
Sous-apprentissage (<i>Underfitting</i>)	Réseau trop simple ou entraînement trop court	Mauvaises performances aussi bien en entraînement qu'en test.
Sur-apprentissage (<i>Overfitting</i>)	Réseau trop complexe ou trop peu de données	Excellentes performances en entraînement, chute sévère en test.

Le découpage systématique du jeu de données en un ensemble d'**entraînement** et un ensemble

de **test** (jamais utilisé pendant l'apprentissage) permet de détecter ces phénomènes et de mesurer objectivement la généralisation du modèle.

1.6 Les différentes dimensions du coût en apprentissage profond

En apprentissage profond industriel, le terme « coût » recouvre trois réalités distinctes qu'il convient de ne pas confondre :

#	Type de coût	Nature	Exemples concrets
1	Coût de la donnée	Acquisition et annotation du jeu de données	Photographe professionnel, expert annotateur humain, équipement haute résolution.
2	Coût de calcul	Temps GPU nécessaire à l'entraînement	Location d'instances cloud, consommation électrique, durée d'entraînement.
3	Coût de déploiement	Inférence en production	Taille du modèle, latence de prédiction, mémoire requise.

Ce projet se concentre prioritairement sur le **Coût de la donnée** (type 1), qui représente le verrou économique principal dans la plupart des applications industrielles réelles. La métrique de **Crédit d'Annotation (CA)**, définie au Chapitre 1, permet de quantifier ce coût de manière rigoureuse et indépendante du matériel utilisé.

Résumé des notions clés

Un réseau de neurones apprend en ajustant itérativement ses **poids** pour minimiser une **fonction de perte**, via la **rétropropagation du gradient**. La qualité de cet apprentissage dépend des **hyperparamètres**, du **volume** et de la **qualité** des données d'entraînement. L'enjeu central de ce projet est de démontrer qu'il est possible de réduire le **coût d'acquisition des données** sans sacrifier les performances de généralisation.

Fondation et construction de l'environnement Multi-Fidélité

Les choix et la mise en place de l'environnement de travail ont été guidés par des contraintes matérielles et par la nécessité de disposer de données adaptées à la simulation d'une approche multi-fidélité.

2.1 Environnement de travail et architecture hybride

Le projet a été mené en **deux phases matérielles successives**. Une première phase de **prototypage** s'est appuyée sur une architecture hybride déportée (VS Code en local + Google Colab pour le GPU), pensée pour allier faible coût et mise en route immédiate. Une seconde phase de **passage à l'échelle** a migré l'ensemble du pipeline vers un **serveur GPU dédié**, rendu nécessaire par le volume final des expériences : $3 \text{ jeux de données} \times (3 \text{ baselines} + 4 \text{ stratégies}) \times 3 \text{ graines aléatoires}$, auxquels s'ajoute une optimisation d'hyperparamètres (Optuna) par stratégie et par dataset.

Phase 1 — Prototypage sur architecture hybride (VS Code + Google Colab)

Outil	Rôle	Détail
VS Code	IDE local	Écriture du code, gestion des terminaux et suivi de projet
Google Colab	Calcul distant	GPU NVIDIA T4 cloud, connecté directement à VS Code via extension sécurisée
Google Drive	Stockage persistant	Monté à chaque session ; héberge le dataset et sauvegarde les <i>checkpoints</i> par époque
GitHub	Versioning	Contrôle de version et sauvegarde du code source expérimental

Un script de test d'intégration a été développé pour valider cette synergie. Il a mis en évidence un **goulot d'étranglement réseau (I/O Bottleneck)** lors de la lecture de milliers de fichiers via le protocole FUSE de Drive, résolu par l'extraction locale d'archives ZIP directement sur le SSD de l'instance Colab.

Phase 2 — Passage à l'échelle sur serveur GPU NVIDIA V100S

Les contraintes propres à Google Colab — sessions limitées dans le temps, déconnexions, GPU T4 partagé et non garanti — sont devenues bloquantes dès lors qu'il a fallu enchaîner des dizaines d'entraînements longs (multi-seed et recherches Optuna). Le projet a donc migré vers un **serveur**

de calcul dédié équipé d'un GPU **NVIDIA Tesla V100S**, nettement plus rapide et stable, et surtout disponible sans interruption.

Outil	Rôle	Détail
Serveur GPU V100S	Calcul dédié	GPU NVIDIA Tesla V100S (Tensor Cores), exécution ininterrompue des entraînements lourds
Apache Guacamole	Accès distant	Passerelle de bureau distant dans le navigateur (sans client lourd), pour piloter le serveur depuis n'importe quel poste
Conda (env pf22)	Environnement	Dépendances isolées (PyTorch + CUDA, Optuna, W&B) reproductibles sur le serveur
tmux + nbconvert	Exécution <i>headless</i>	Les notebooks sont exécutés en série (<code>jupyter nbconvert --execute</code>) dans une session <code>tmux</code> qui survit à la déconnexion ; W&B en mode <i>offline</i> pour ne jamais bloquer

Concrètement, l'intégralité des notebooks est lancée par un script unique (`run_all.sh`) à l'intérieur d'un `tmux` : on peut alors fermer le poste client sans interrompre le calcul, se reconnecter via Guacamole pour suivre l'avancement, puis récupérer les résultats (fichiers JSON et figures) une fois le *run* terminé. C'est sur cette infrastructure qu'ont été produits tous les résultats chiffrés de ce rapport.

2.2 Sélection du jeu de données

Le jeu de données doit être suffisamment difficile pour que la dégradation visuelle impacte mesurément les performances du réseau. Six candidats ont été évalués :

Dataset	Images	Avantages	Inconvénients
Imagenette	13 000	Standard académique, itérations rapides	Trop « facile » pour ResNet-18 ($\approx 95\%$ sans dégradation)
STL-10	105 000	Grand jeu non étiqueté (100k images)	Résolution native 96×96 trop faible
Oxford Pets	7 400	Classification fine-grained, très sensible aux dégradations	Volume insuffisant, nécessite un pré-entraînement lourd
Animals-10	26 179	Haute résolution, volume idéal, proche des conditions réelles	Bruit de labels et filigranes (peuvent être corrigés)
Imagewoof	12 954	10 races de chiens visuellement très proches, haute résolution	Volume environ trois fois inférieur à Animals-10, risque de sur-apprentissage
Intel	$\approx 17\,000$	6 classes de scènes (naturelles et urbaines), tâche différente des animaux	Résolution native modérée (150 px), classes parfois ambiguës (<i>glacier/montagne</i>)

Choix retenu : Animals-10, Imagewoof et Intel (triple validation)

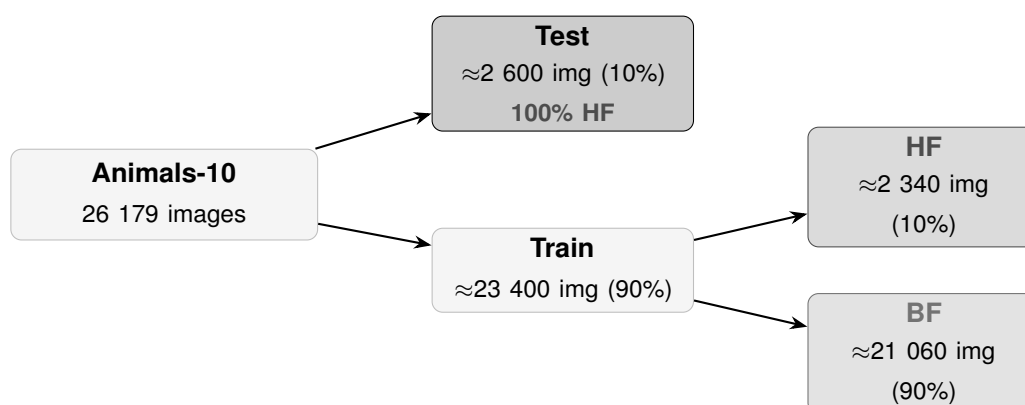
Afin de garantir que les conclusions tirées sur les stratégies multi-fidélité ne sont pas un artefact propre à un dataset particulier, **trois jeux de données ont été retenus** et le protocole expérimental complet est exécuté sur chacun :

- **Animals-10** (26 179 images, 10 classes d'animaux) — dataset principal d'expérimentation. Son volume permet un entraînement *from scratch* stable avec ResNet-18, sa haute résolution offre une large plage dynamique pour le pipeline de dégradation, et son caractère communautaire le rapproche des conditions industrielles réelles.
- **Imagewoof** (12 954 images, 10 races de chiens) — dataset de **validation croisée fine-grained**. Sous-ensemble d'ImageNet plus difficile (classification de classes visuellement très proches) et nettement plus petit en volume, il met les stratégies sous pression.
- **Intel** ($\approx 17\,000$ images, 6 classes de scènes) — dataset de **validation sur une tâche de nature différente** (reconnaissance de scènes naturelles et urbaines plutôt que d'objets). Il vérifie que les conclusions ne dépendent pas du *type* de problème de vision considéré.

Une stratégie qui reste la meilleure sur les trois datasets — volumes, granularités et natures de tâche différents — peut alors être considérée comme **robuste** et non liée aux spécificités d'un seul jeu de données. Les trois datasets suivent strictement le même pipeline (répartition test 10 % / train 90 %, train découpé en 10 % HF + 90 % BF, même dégradation, ResNet-18 *from scratch*, 20 époques, AMP FP16).

2.3 Découpage et répartition des données

Afin d'étudier l'impact de la qualité des données sur les performances d'apprentissage, un environnement expérimental a été construit : découpage asymétrique du dataset, pipeline de dégradation et métrique de coût. Le schéma ci-dessous illustre le découpage du jeu Animals-10.



Principe clé

Le jeu de **test est exclusivement HF** pour toutes les expériences : l'évaluation doit mesurer la capacité de généralisation sur des données de haute qualité, quel que soit le domaine utilisé à l'entraînement.

2.4 Pipeline de dégradation de la qualité

Les images de Basse Fidélité ne sont **pas** stockées dégradées sur le disque : elles sont conservées propres, et la dégradation est appliquée **à la volée** au chargement, de façon **strictement identique à l'entraînement et à l'évaluation**. Une unique fonction de dégradation, centralisée dans le module `src/degradation.py`, est partagée par toutes les expériences, ce qui élimine tout décalage de domaine parasite entre le train BF et le test BF. La dégradation est en outre **déterministe par image** (graine dérivée de l'indice), donc reproductible d'une époque et d'un *run* à l'autre.

La dégradation enchaîne **trois transformations** :

#	Transformation	Description	Effet simulé
1	Sous-échantillonnage (<i>Downsampling</i>)	Réduction à 64×64 px puis retour à 224×224 px par interpolation bilinéaire avec anticrénelage (<i>antialias</i>)	Perte d'information : image floue et pixélisée
2	Bruit Gaussien	Ajout d'un bruit aléatoire (grain, $\sigma = 0,15$) aux tenseurs de l'image	Simulation du bruit ISO des capteurs électroniques
3	Compression JPEG	Ré-encodage JPEG à qualité réduite (<i>quality</i> = 60)	Artefacts de blocs et quantification d'un stockage bas de gamme

Cohérence train/test du domaine BF

Train BF et Test BF passent désormais par **exactement** le même pipeline (mêmes interpolation, bruit σ et qualité JPEG), via un module unique (`src/degradation.py`). La précision sur Test BF mesure donc la robustesse au domaine appris, sans biais d'implémentation. Les paramètres (résolution intermédiaire, σ , qualité JPEG) sont exposés afin de permettre l'étude de l'*intensité* de la dégradation.

2.5 Définition de la métrique de coût

Trois coûts sont désormais distingués explicitement et reportés séparément :

- le **coût de la donnée** (*Crédit d'Annotation*, CA) : coût d'**acquisition** du jeu de données, payé **une seule fois**, $= \sum (\text{images distinctes} \times \text{coût unitaire})$. Indépendant du nombre d'époques ;
- le **coût de calcul** : nombre total d'images traitées = taille du train $\times$ époques (*non pondéré*), indicateur agnostique au matériel ;
- le **coût total** : images traitées **pondérées par leur coût unitaire** $= \sum_{\text{époques}} (\text{images} \times \text{coût unitaire})$. Une image HF y pèse $\times 10$ à *chaque époque* ; c'est la métrique qui valorise le plus le recours aux données BF bon marché par les stratégies multi-fidélité.

Le coût unitaire d'acquisition d'une image dépend de sa **résolution** (qualité du capteur, soin de la prise de vue, stockage), donc approximativement du nombre de pixels (résolution au carré). Le bruit gaussien et la compression JPEG sont considérés comme un post-traitement sans surcoût d'acquisition. On pose :

$$C(d) = C_{\min} + (C_{HF} - C_{\min}) \left(\frac{d}{224} \right)^2$$

où d est la résolution effective de l'image. Les paramètres sont **calibrés** sur deux ancres : une image pleine résolution ($d = 224$) coûte $C_{HF} = 10$ CA, et l'image Basse Fidélité canonique ($d = 64$) coûte

≈ 1 CA, ce qui fixe $C_{min} \approx 0,2$ (coût plancher de toute image). Ce modèle rend le ratio $\approx 10:1$ **justifié** (et non plus arbitraire) par la quantité d'information acquise.

Type d'image	Coût C	Justification
Haute Fidélité ($d = 224$)	10 CA	Expert humain ou équipement onéreux (pleine résolution)
Basse Fidélité ($d = 64$)	≈ 1 CA	Collecte automatisée, faible résolution, pré-étiquetage web

Exemple chiffré (Animals-10). Le pool d'entraînement comporte 2 352 images HF et 21 213 images BF. Le **coût donnée** (acquisition, payé une fois) vaut donc :

$$\underbrace{2\,352 \times 10}_{\text{HF}} + \underbrace{21\,213 \times 1}_{\text{BF}} = \mathbf{44\,733\,CA}.$$

C'est le coût donnée commun à toute méthode exploitant ce pool complet (*baseline* Mixte comme stratégies multi-fidélité). Ces méthodes ne se départagent alors que sur le **coût total** (images vues pondérées, sommées sur les époques), qui dépend du nombre de passages effectués sur les images HF coûteuses : c'est précisément ce levier que les stratégies multi-fidélité exploitent. Les deux colonnes « Coût données » et « Coût total » des tableaux de résultats reprennent directement ces deux grandeurs.

Trois coûts complémentaires

Le **coût donnée** (CA) mesure l'*acquisition*, compté **une seule fois** : deux méthodes exploitant le même pool HF+BF ont donc le **même coût donnée**. Elles se distinguent sur le **coût de calcul** (images vues) et surtout le **coût total** (images vues pondérées par leur coût unitaire) : c'est là que les stratégies multi-fidélité créent de la valeur, en remplaçant des passages sur des images HF coûteuses par des passages sur des images BF bon marché. Le temps de calcul brut, dépendant du matériel (charge serveur, allocation GPU, I/O), est suivi à titre indicatif mais n'est pas la métrique principale.

2.6 Sélection et justification de l'architecture neuronale

Pour que la comparaison entre nos différents jeux de données (HF, BF, Mixte) soit scientifiquement valide, l'architecture doit rester strictement identique. Plusieurs architectures standards de vision par ordinateur ont été évaluées :

Architecture	Paramètres	Points forts	Limites / Pertinence
ResNet-18 ✓	≈11 M	Standard académique, connexions résiduelles, robuste au bruit.	Pertinence excellente — compromis idéal.
MobileNetV2	≈3,4 M	Ultra-léger, économe en VRAM, itérations rapides.	S'effondre sur données fortement bruitées.
EfficientNet-B0	≈5,3 M	Ratio précision/paramètres optimisé mathématiquement.	Difficile à faire converger <i>from scratch</i> .
ViT-Tiny	≈5 M	Architecture Transformer, vision globale.	Sur-apprend massivement sans pré-entraînement.

Architecture retenue : ResNet-18 (Entraîné *from scratch*)

Ses ≈11 millions de paramètres lui confèrent une capacité d'abstraction suffisante pour extraire des caractéristiques à travers le bruit. Il a été délibérément choisi de ne pas utiliser de poids pré-entraînés (`weights=None`) afin de mesurer le véritable impact qualitatif de notre jeu de données Animals-10 sur l'apprentissage, sans biais cognitif préalable. La dernière couche (classifieur) a été adaptée pour générer 10 vecteurs de sortie correspondant à nos 10 classes animales.

2.6.1 Mécanique interne de ResNet-18

L'architecture *Residual Network* (ResNet) a révolutionné l'apprentissage profond en introduisant les **connexions résiduelles** (*skip connections*). Dans un réseau convolutionnel classique profond, le signal du gradient a tendance à s'estomper lors de la phase de rétropropagation (le problème de la disparition du gradient). ResNet contourne ce problème en ajoutant l'entrée d'une couche directement à sa sortie mathématique ($F(x) + x$).

Dans le contexte de données Basse Fidélité (flou et bruit gaussien), cette architecture est cruciale : si une couche convolutionnelle n'arrive pas à extraire d'information pertinente à cause du bruit, la connexion résiduelle permet à l'information globale de l'image de continuer à circuler librement à travers le réseau sans être détruite. La version à 18 couches (ResNet-18) offre le compromis parfait entre expressivité et prévention du sur-apprentissage (*overfitting*) sur notre corpus de ≈25 000 images.

2.6.2 Justification des hyperparamètres et de la fonction de coût

Pour garantir la reproductibilité et la comparabilité des *Baselines*, une configuration stricte d'hyperparamètres a été codée en dur dans notre script d'entraînement (`train_baselines.py`) :

— **Fonction de perte (Loss)** – `CrossEntropyLoss` : Il s'agit du standard mathématique pour les problèmes de classification multi-classes. Cette fonction combine une activation Softmax (pour transformer les sorties brutes du modèle en probabilités de 0 à 1) et une perte log-vraisemblance

négligée (NLL). Elle pénalise fortement le modèle s'il est très confiant sur une mauvaise prédiction, ce qui est idéal pour forcer le réseau à douter sur des images floues.

- **Optimiseur – Adam** : Contrairement à une descente de gradient stochastique classique (SGD), Adam adapte dynamiquement le taux d'apprentissage pour chaque paramètre individuel en utilisant les moments du premier et second ordre des gradients. Cela permet une convergence beaucoup plus rapide et stable, particulièrement utile lors d'un entraînement *from scratch*.
- **Taux d'apprentissage (*Learning Rate*)** : Fixé à $lr = 0.001$. C'est la valeur par défaut recommandée pour l'optimiseur Adam, offrant un pas de descente suffisamment grand pour échapper aux minima locaux lors des premières époques, tout en évitant la divergence.
- **Taille de lot (*Batch Size*)** : Fixée à 64. Cette valeur permet d'optimiser l'utilisation de la mémoire vidéo (VRAM) du GPU, tout en garantissant un gradient stochastique suffisamment lissé pour ne pas rendre l'apprentissage chaotique face au bruit des images BF.
- **Nombre d'époques** : Fixé à 20. Des tests préliminaires ont montré qu'au-delà de 20 époques, la courbe d'apprentissage sur le jeu d'entraînement atteint un plateau asymptotique, et le risque de mémorisation du bruit (*overfitting*) augmente considérablement.

2.6.3 Prétraitement et optimisations matérielles

Afin d'exploiter pleinement le matériel GPU (architecture hybride Colab en prototypage, puis serveur V100S en production), deux optimisations logicielles majeures ont été intégrées à l'implémentation :

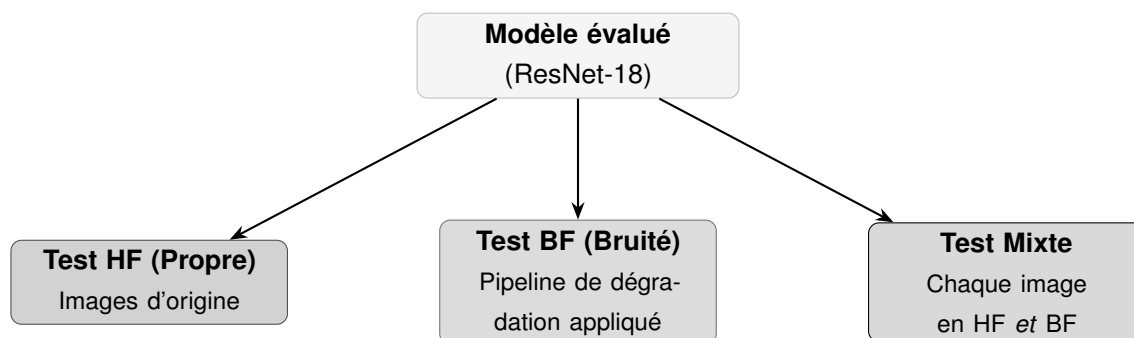
1. **Normalisation des tenseurs** : Avant d'entrer dans le réseau, toutes les images (redimensionnées en 224×224 pixels) subissent une normalisation spatiale basée sur les statistiques du jeu de données ImageNet (Moyennes : $[0.485, 0.456, 0.406]$, Écarts-types : $[0.229, 0.224, 0.225]$). Cette opération centre les données autour de zéro, accélérant la convergence de la descente de gradient.
2. **Précision Mixte Automatique (AMP)** : L'entraînement utilise les modules `torch.amp.autocast` et `GradScaler`. Cette technique exécute certaines opérations mathématiques en virgule flottante 16 bits (FP16) au lieu de 32 bits (FP32). Cela a permis de réduire drastiquement la consommation mémoire et d'accélérer le temps de calcul sur les Tensor Cores du GPU (NVIDIA T4 puis V100S), réduisant fortement la durée d'une époque, sans aucune perte de précision prédictive.

Construction des Modèles de Référence

Avant d'explorer des stratégies d'optimisation complexes, il est indispensable d'établir des **bornes de performances** (supérieure et inférieure) via des *baselines*.

3.1 Stratégie d'évaluation croisée face au décalage de domaine

Un modèle entraîné sur des images floues (domaine BF) peut ne pas généraliser correctement sur des images nettes (domaine HF) — c'est le **Domain Shift**. Pour quantifier ce phénomène, chaque modèle est évalué selon **trois passages distincts** :



Définition du Test Mixte

Le **Test Mixte** agrège les deux domaines : chaque image du jeu de test est évaluée **une fois en HF et une fois en BF**, et l'on rapporte la précision sur l'ensemble de ces prédictions ($2N$ prédictions pour N images). Les deux passages portant sur les mêmes images, cette valeur est **exactement** la moyenne des précisions Test HF et Test BF : c'est un estimateur équilibré et déterministe du comportement mixte.

3.2 Définition des modèles de référence

Les volumes et coûts indiqués ci-dessous correspondent au dataset principal **Animals-10** (coût donnée d'acquisition, payé une fois) ; les ordres de grandeur sont analogues sur Imagewoof et Intel.

#	Modèle	Images	Coût donnée	Hypothèse de comportement
1	Premium (Train HF uniquement)	≈2 352 HF	23 520 CA	Excellentes performances sur Test HF. Effondrement prévu sur Test BF (aucune robustesse au bruit).
2	Low-Cost (Train BF uniquement)	≈21 213 BF	21 213 CA	Très performant sur Test BF. Généralisation sur Test HF dépendra de la capacité à extrapoler des contours nets depuis des signaux flous.
3	Naïf Mixte (Borne supérieure)	≈23 565 HF+BF	44 733 CA	Meilleures performances globales (Test Mixte). Sert de borne supérieure absolue. L'objectif des phases suivantes est d'atteindre ce niveau à coût réduit.

3.3 Stratégie d'implémentation logicielle

Afin d'éliminer tout biais d'implémentation, le code a été factorisé en un **unique script d'entraînement paramétrable**. Ce script accepte un argument de mode (HF, BF, ou MIXTE), charge dynamiquement le sous-ensemble correspondant, procède à l'entraînement et exécute automatiquement la séquence des trois tests d'évaluation croisée.

3.4 Analyse des résultats

Les trois modèles de référence ont été entraînés sur 20 époques, sur chacun des **trois datasets** retenus et pour **trois graines aléatoires** (42, 1, 2). Les résultats ci-dessous sont reportés en **moyenne** $\pm$ **écart-type** sur ces trois graines, ce qui rend les conclusions robustes au hasard d'initialisation. Deux coûts sont distingués (cf. §2.5) : le **coût donnée** (acquisition, payé une fois) et le **coût total** (calcul pondéré, sommé sur les époques).

Modèle	Test HF	Test BF	Test Mixte	Coût donnée	Coût total
<i>Animals-10</i>					
BL 1 (HF)	54,3 ± 2,2	27,8 ± 0,8	41,1 ± 1,1	23 520	470 400
BL 2 (BF)	61,3 ± 6,3	71,0 ± 1,5	66,2 ± 2,7	21 213	424 260
BL 3 (Mixte)	74,0 ± 3,0	72,9 ± 1,1	73,4 ± 2,0	44 733	894 660
<i>Imagewoof</i>					
BL 1 (HF)	25,0 ± 4,1	22,1 ± 1,4	23,5 ± 2,6	8 990	179 800
BL 2 (BF)	42,7 ± 2,8	47,0 ± 1,6	44,9 ± 2,1	8 126	162 520
BL 3 (Mixte)	50,3 ± 2,5	48,8 ± 2,3	49,6 ± 2,4	17 116	342 320
<i>Intel</i>					
BL 1 (HF)	75,8 ± 2,2	47,0 ± 0,8	61,4 ± 1,1	14 020	280 400
BL 2 (BF)	78,7 ± 2,7	81,9 ± 1,4	80,3 ± 1,4	12 632	252 640
BL 3 (Mixte)	82,8 ± 1,2	82,1 ± 1,2	82,5 ± 1,2	26 652	533 040

L'analyse de ces métriques sur **Animals-10** (dataset principal) permet de tirer trois conclusions majeures concernant le comportement du réseau ResNet-18 face à nos données :

- **L'effondrement face au bruit (*Domain Shift*)** : La Baseline 1, entraînée exclusivement sur des données parfaites, obtient un score de 54,3 % sur le test HF. Cependant, confrontée au domaine BF (flou et bruit gaussien), sa précision s'effondre à 27,8 %. Ce résultat prouve l'incapacité d'un modèle "Premium" classique à généraliser face à des perturbations non rencontrées lors de l'apprentissage.
- **Le paradoxe de la quantité** : La Baseline 2 possède un coût donnée inférieur à la Baseline 1 (21 213 CA contre 23 520 CA) mais la surpasse **y compris sur les images nettes** (61,3 % vs 54,3 % en HF). Cela démontre qu'en apprentissage profond, l'immense diversité contenue dans $\approx 21\,200$ images floues prime sur la perfection visuelle de seulement $\approx 2\,350$ images.
- **La validation de la borne supérieure** : La Baseline 3 confirme son statut théorique. En ayant appris simultanément sur le volume massif (BF) et sur les détails discriminants (HF), elle atteint une précision de 74,0 % sur le test de haute fidélité. Toutefois, son coût (donnée comme total) est nettement plus élevé que celui des autres baselines.

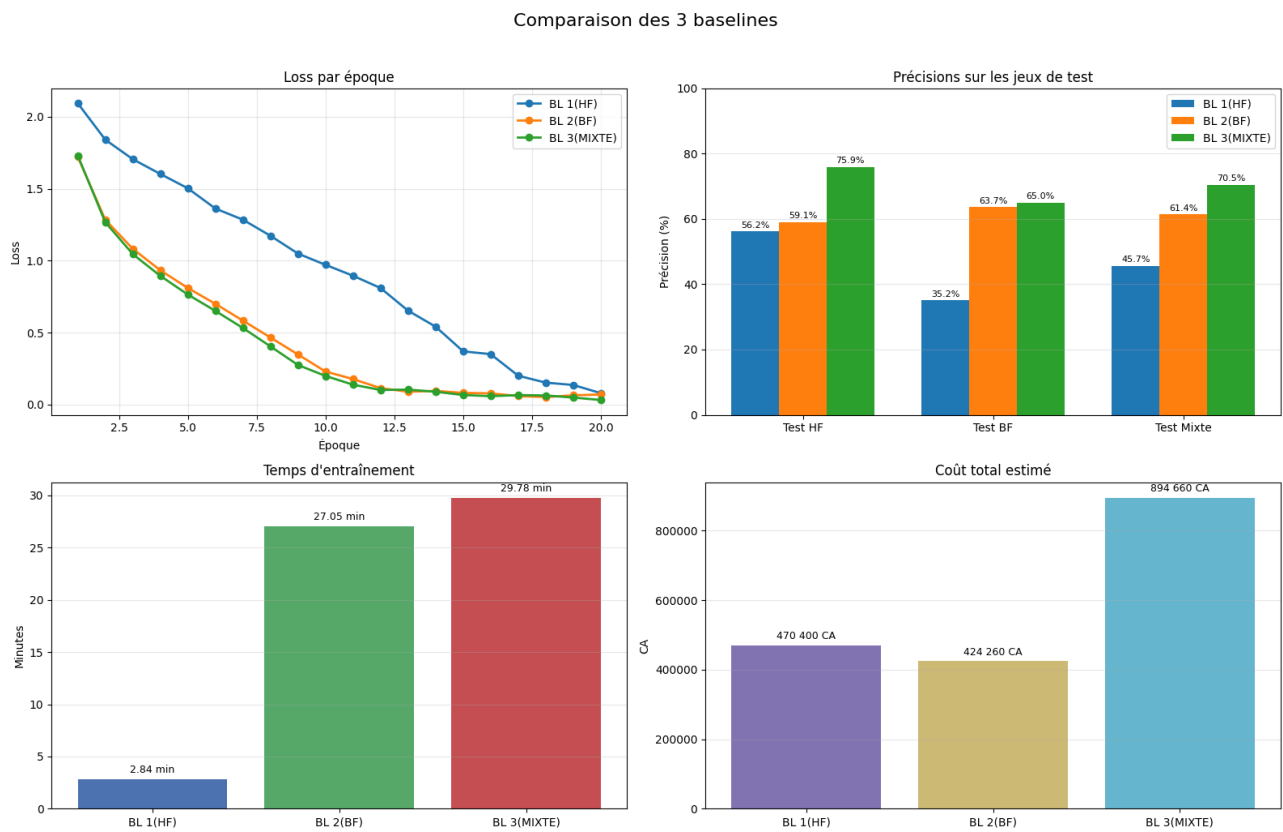


FIGURE 3.1 – Comparaison visuelle des trois baselines sur **Animals-10** (BL 1(HF), BL 2(BF), BL 3(MIXTE)) : loss par époque, précisions de test, temps d'entraînement et coût total estimé.

Vérification sur Imagewoof

Réexécutées sur Imagewoof, les trois baselines confirment les conclusions tirées sur Animals-10, malgré des valeurs absolues nettement inférieures (Imagewoof = classification *fine-grained* de races canines, $\approx 3\times$ moins d'images d'entraînement) :

- **Domain Shift confirmé** : la BL 1 passe de 25,0 % (HF) à 22,1 % (BF) ; la chute est moins spectaculaire qu'ailleurs uniquement parce que la BL 1 est déjà très faible en HF sur ce problème difficile.
- **Paradoxe de la quantité confirmé** : la BL 2 (coût donnée 8 126 CA) bat la BL 1 (8 990 CA) sur les trois tests, y compris HF (42,7 % vs 25,0 %), avec un coût inférieur.
- **Borne supérieure validée** : la BL 3 atteint 50,3 % sur HF, meilleur score, pour un coût donnée double de BL 1.

La hiérarchie **BL 3 > BL 2 > BL 1** est strictement conservée, ce qui valide la robustesse des conclusions indépendamment du dataset.

Comparaison des 3 baselines — Imagewoof

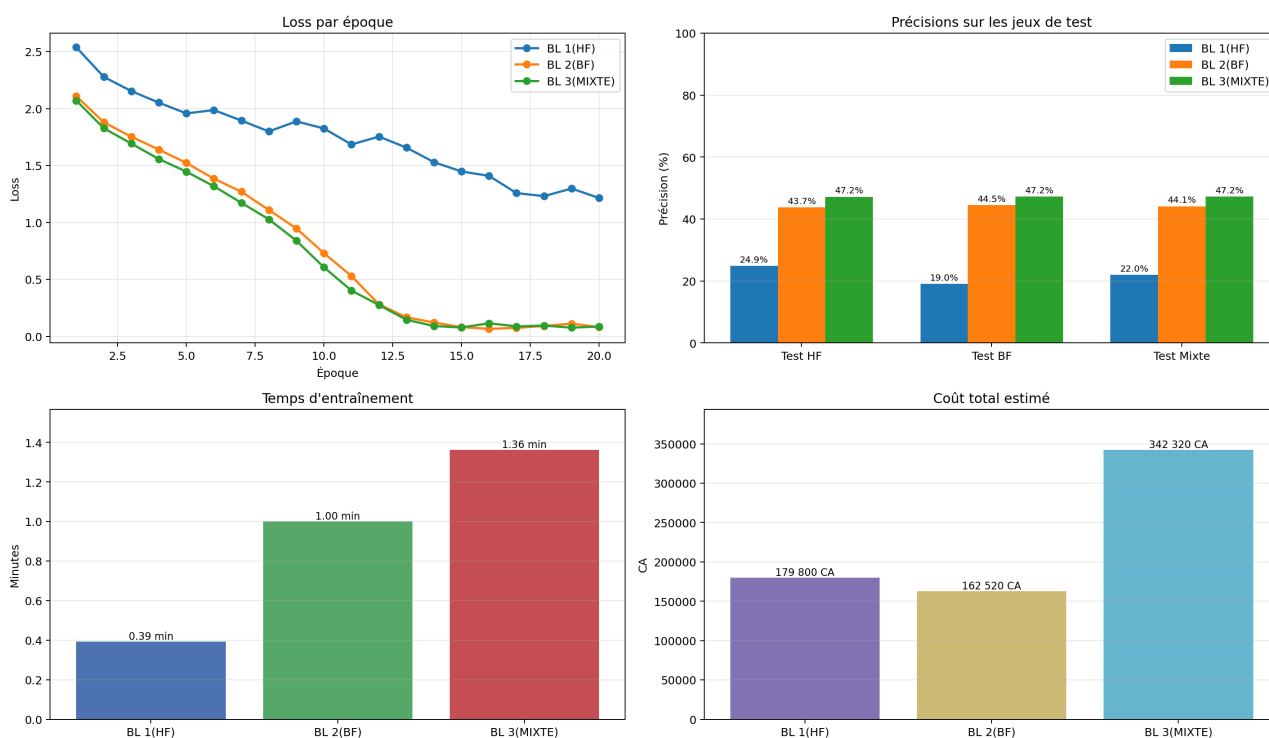


FIGURE 3.2 – Comparaison visuelle des trois baselines sur **Imagewoof** : précisions de test, temps d'entraînement et coût total estimé. Les valeurs absolues sont plus basses qu'Animals-10 (classification *fine-grained*) mais la hiérarchie **BL 3 > BL 2 > BL 1** est strictement identique.

Vérification sur Intel

Sur Intel (6 classes de scènes, tâche de nature différente), les valeurs absolues sont nettement plus hautes — la reconnaissance de scènes est plus facile que la classification *fine-grained* — mais la hiérarchie reste identique :

- **Domain Shift très marqué** : la BL 1 chute de 75,8 % (HF) à 47,0 % (BF), soit −28,8 points, l’effondrement le plus net des trois datasets.
- **Paradoxe de la quantité confirmé** : la BL 2 (coût donnée 12 632 CA) dépasse la BL 1 (14 020 CA) sur les trois tests, HF compris (78,7 % vs 75,8 %), et se montre remarquablement robuste sur BF (81,9 %).
- **Borne supérieure validée** : la BL 3 plafonne le panel (82,8 % HF, 82,5 % Mixte) pour le coût le plus élevé.

La **hiérarchie BL 3 > BL 2 > BL 1** se vérifie donc sur les **trois** datasets, malgré des volumes, granularités et natures de tâche très différents.

CHAPITRE 4

Stratégies Multi-Fidélité

Ce chapitre présente les stratégies multi-fidélité, toutes orientées vers un **objectif HF prioritaire** : maximiser la performance sur données propres (Test HF) tout en exploitant le volume d'images BF bon marché pour réduire le coût. Quatre stratégies sont étudiées, chacune sur les trois datasets et en multi-seed (moyenne $\pm$ écart-type) :

- **Stratégie 1 — Transfer Learning séquentiel** (BF $\rightarrow$ HF) : pré-entraînement sur le volume BF puis fine-tuning sur le sous-ensemble HF ;
- **Stratégie 2 — Co-training pondéré** : les deux domaines sont vus simultanément dans chaque lot, avec une pondération explicite de la perte ;
- **Stratégie 3 — Curriculum Learning progressif** : la proportion d'images HF augmente par paliers au fil des époques (du tout-BF vers le tout-HF) ;
- **Stratégie 4 — Elastic Weight Consolidation (EWC)** : variante du transfert séquentiel où une pénalité de Fisher protège, pendant le fine-tuning HF, les poids importants pour le domaine BF (lutte contre l'oubli catastrophique).

Chaque stratégie est en outre déclinée en une version dont les hyperparamètres ont été optimisés par **Optuna** ; l'analyse détaillée de ces gains fait l'objet d'un chapitre dédié, les sections ci-dessous portant sur les configurations de base.

4.1 Stratégie 1 – Transfer Learning Séquentiel (BF $\rightarrow$ HF)

4.1.1 Protocole

La stratégie retenue repose sur deux phases d'entraînement consécutives avec la même architecture ResNet-18 :

1. **Pré-entraînement BF** : apprentissage sur le volume principal de données dégradées pour acquérir une représentation robuste et générique des classes.
2. **Fine-tuning HF** : spécialisation sur un sous-ensemble HF plus petit, afin de récupérer les détails discriminants de haute fidélité.

Configuration expérimentale utilisée :

Paramètre	Valeur	Commentaire
Architecture	ResNet-18	Entraînée <i>from scratch</i> , tête de classification 10 classes
Optimiseur	Adam	Identique aux baselines pour conserver la comparabilité
Batch size	64	Compromis stabilité/vitesse sur GPU V100S
Époques BF	10	21 213 images BF (332 batches/époque)
Époques HF	10	1 881 images HF train + 471 images HF val
Learning rates	0,001 puis 0,0003	Plus faible en fine-tuning pour stabiliser l'adaptation
Accélération	AMP (FP16)	autocast + GradScaler

4.1.2 Résultats quantitatifs obtenus

Les résultats finaux de l'Étape 1 sont résumés ci-dessous pour les trois datasets (moyenne $\pm$ écart-type sur 3 graines). Dans cette étude, le critère prioritaire est la performance sur **Test HF**.

Dataset	Test HF	Test BF	Test Mixte	Coût donnée	Coût total
Animals-10	77,7 $\pm$ 0,6	68,3 $\pm$ 0,5	73,0 $\pm$ 0,3	44 733	400 230
Imagewoof	51,0 $\pm$ 1,6	44,7 $\pm$ 1,3	47,9 $\pm$ 1,5	17 116	153 160
Intel	86,1 $\pm$ 0,4	79,1 $\pm$ 1,4	82,6 $\pm$ 0,7	26 652	238 420

Lecture rapide des courbes d'entraînement (Animals-10, graine 42)

La phase BF converge rapidement en *train loss*, alors que la validation HF fluctue fortement pendant le pré-entraînement. Après transition vers HF, la précision validation HF bondit immédiatement autour de 77–78 %, puis se stabilise ; la *val loss* remonte légèrement en fin de fine-tuning, signalant un début de sur-apprentissage. Sur les 3 graines, le score final HF est très stable (77,7 $\pm$ 0,6 %).

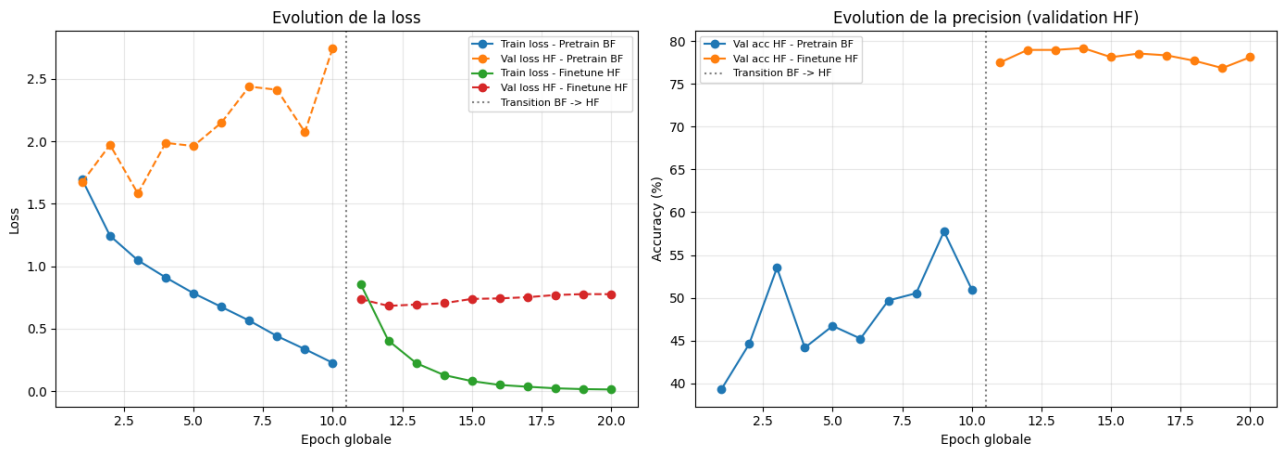


FIGURE 4.1 – Évolution de la loss et de la précision validation HF pendant le pré-entraînement BF puis le fine-tuning HF, sur **Animals-10**.

Strategie 1 (BF -> HF) - Imagewoof

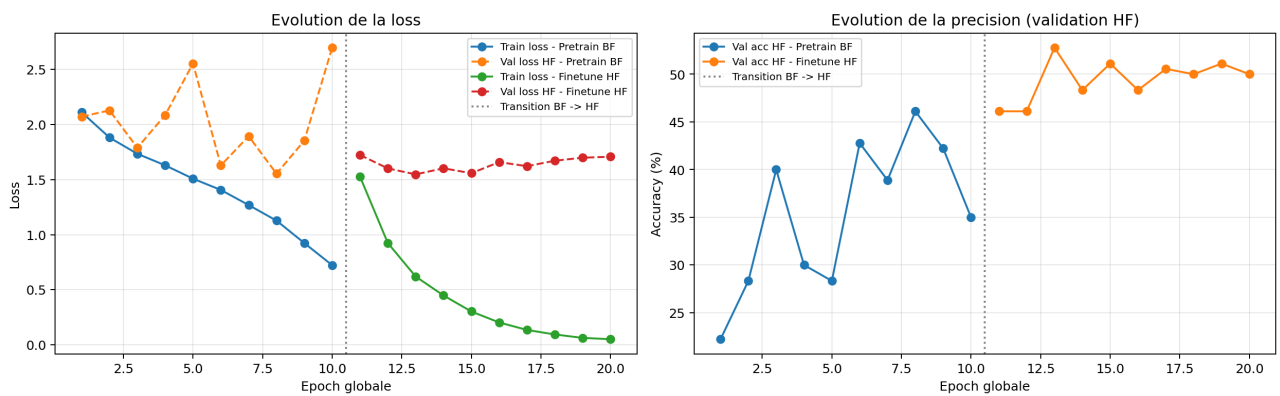


FIGURE 4.2 – Évolution de la loss et de la précision validation HF pendant le pré-entraînement BF puis le fine-tuning HF, sur **Imagewoof**.

4.1.3 Comparaison avec les baselines du Chapitre 3

Modèle	HF	BF	Mixte	Coût donnée	Coût total
<i>Animals-10</i>					
BL 1 (HF)	54,3 ± 2,2	27,8 ± 0,8	41,1 ± 1,1	23 520	470 400
BL 2 (BF)	61,3 ± 6,3	71,0 ± 1,5	66,2 ± 2,7	21 213	424 260
BL 3 (Mixte)	74,0 ± 3,0	72,9 ± 1,1	73,4 ± 2,0	44 733	894 660
Stratégie 1	77,7 ± 0,6	68,3 ± 0,5	73,0 ± 0,3	44 733	400 230
<i>Imagewoof</i>					
BL 1 (HF)	25,0 ± 4,1	22,1 ± 1,4	23,5 ± 2,6	8 990	179 800
BL 2 (BF)	42,7 ± 2,8	47,0 ± 1,6	44,9 ± 2,1	8 126	162 520
BL 3 (Mixte)	50,3 ± 2,5	48,8 ± 2,3	49,6 ± 2,4	17 116	342 320
Stratégie 1	51,0 ± 1,6	44,7 ± 1,3	47,9 ± 1,5	17 116	153 160
<i>Intel</i>					
BL 1 (HF)	75,8 ± 2,2	47,0 ± 0,8	61,4 ± 1,1	14 020	280 400
BL 2 (BF)	78,7 ± 2,7	81,9 ± 1,4	80,3 ± 1,4	12 632	252 640
BL 3 (Mixte)	82,8 ± 1,2	82,1 ± 1,2	82,5 ± 1,2	26 652	533 040
Stratégie 1	86,1 ± 0,4	79,1 ± 1,4	82,6 ± 0,7	26 652	238 420

Comparaisons clés (Animals-10) :

- **Objectif principal HF atteint** : la stratégie 1 obtient **77,7 % sur Test HF**, supérieure à **toutes les baselines** (BL 3 = 74,0 %, BL 2 = 61,3 %, BL 1 = 54,3 %), et avec le plus faible écart-type (0,6), donc la plus stable.
- **Vs BL 3 (Mixte)** : à **coût donnée identique** (44 733 CA, même pool HF+BF), elle dépasse BL 3 sur HF (+3,7 points) tout en réduisant le **coût total** de ≈ 55 % (894 660 → 400 230 CA), car le HF coûteux n'est vu qu'en phase de fine-tuning.
- **Compromis BF modéré** : la précision BF (68,3 %) reste légèrement inférieure à BL 2 (71,0 %) et BL 3 (72,9 %), compromis cohérent avec la priorité explicitement fixée sur la qualité HF.

Vérification sur Imagewoof et Intel

La Stratégie 1 confirme son rang sur les deux datasets de validation, dépassant les trois baselines sur Test HF :

- **Imagewoof** : 51,0 % HF, soit le meilleur score parmi baselines et S1 (BL 3 = 50,3 %). L'avantage HF sur BL 3 est ténu (+0,7 point) mais le **coût total est divisé par $\approx 2,2$** (153 160 vs 342 320 CA) à coût donnée identique — l'intérêt de la séparation BF $\rightarrow$ HF est ici surtout économique.
- **Intel** : 86,1 % HF, nettement devant BL 3 (82,8 %), +3,3 points, là encore pour un coût total réduit de plus de moitié (238 420 vs 533 040 CA).

La hiérarchie **Stratégie 1 > BL 3 > BL 2 > BL 1** sur Test HF est **strictement identique sur les trois datasets**, ce qui confirme que la dominance du transfer learning séquentiel sur les baselines n'est pas un artefact d'un jeu de données particulier.

4.1.4 Analyse : ce qui a marché, ce qui a moins marché, et enseignements

- **Ce qui a bien marché (HF)** : la phase BF fournit une initialisation robuste, puis le fine-tuning HF spécialise efficacement le modèle ; le gain final en Test HF est net et place la stratégie 1 en tête.
- **Limite observée** : la spécialisation HF entraîne une perte partielle de robustesse BF (*domain forgetting*), ce qui était attendu dans une optimisation orientée domaine propre.
- **Signal de sur-apprentissage léger** : en fin de fine-tuning, la *train loss* continue de baisser tandis que la *val loss* HF se dégrade légèrement ; un *early stopping* piloté par Test/Val HF améliorerait encore la stabilité.
- **Leçon principale** : pour un objectif centré sur la performance HF, le transfer learning séquentiel offre le meilleur compromis observé entre précision cible et coût d'annotation.

Conclusion Étape 1 (objectif HF prioritaire) : la stratégie BF $\rightarrow$ HF est validée comme meilleure option actuelle pour maximiser la performance sur données propres. L'Étape 2 (co-training pondéré) visera désormais à conserver ce niveau HF tout en regagnant de la robustesse sur BF.

4.1.5 Améliorations possibles de la méthode (Stratégie 1)

Les résultats obtenus montrent que la stratégie séquentielle est déjà très compétitive sur HF, mais plusieurs leviers peuvent encore être optimisés.

- **Nombre d'époques BF et HF** : modifier la durée relative pré-entraînement/fine-tuning (par ex. 5/15, 8/12, 12/8) peut déplacer le compromis robustesse BF vs spécialisation HF. *Comment optimiser* : effectuer une recherche en grille sur le couple (E_{BF}, E_{HF}) et sélectionner la configuration maximisant Test HF.
- **Learning rates par phase** : un LR trop élevé en fine-tuning peut détruire des représentations utiles, trop faible peut sous-adapter HF. *Comment optimiser* : tester plusieurs couples (lr_{BF}, lr_{HF})

avec schémas de décroissance (Cosine, StepLR, ReduceLROnPlateau).

- **Politique de gel/dégel des couches** : geler partiellement le backbone au début du fine-tuning peut limiter l'oubli BF tout en spécialisant la tête. *Comment optimiser* : comparer *full fine-tuning* vs *head-only* puis dégel progressif bloc par bloc.
- **Split HF train/validation** : la proportion 80/20 peut être remplacée (70/30, 85/15) pour stabiliser la sélection de modèle. *Comment optimiser* : valider sur plusieurs seeds et retenir la configuration la plus stable en variance de score HF.
- **Batch size** : influe sur la stabilité du gradient et la généralisation. *Comment optimiser* : explorer 32/64/96 (selon VRAM) en ajustant le LR selon une règle de mise à l'échelle.
- **Régularisation** : ajouter *weight decay*, *dropout* ou *label smoothing* peut réduire le sur-apprentissage en fin de phase HF. *Comment optimiser* : balayage de plages courtes (ex. $\text{weight decay} \in [10^{-5}, 10^{-3}]$, $\text{label smoothing} \in [0.0, 0.1]$).
- **Augmentations de données** : RandAugment, ColorJitter léger, RandomErasing peuvent améliorer la robustesse sans dégrader HF si bien dosés. *Comment optimiser* : ablations une augmentation à la fois puis combinaison des plus bénéfiques.
- **Early stopping et checkpointing** : arrêter sur la meilleure validation HF évite la dégradation tardive. *Comment optimiser* : utiliser une patience (3 à 5 époques) et conserver systématiquement le meilleur checkpoint HF.
- **Architecture backbone** : ResNet-18 est un bon compromis, mais ResNet-34 ou EfficientNet-B0 peuvent améliorer la représentation. *Comment optimiser* : comparer à budget de coût constant et même protocole d'évaluation.
- **Fonction de coût avancée** : Focal Loss ou marges additives peuvent mieux séparer des classes ambiguës en HF. *Comment optimiser* : tests contrôlés en remplaçant uniquement la loss, avec mêmes données et mêmes hyperparamètres de base.

4.2 Stratégie 2 – Co-training par Batch avec Pondération (Reweighting)

4.2.1 Protocole

Contrairement à la stratégie séquentielle BF $\rightarrow$ HF, cette approche entraîne le modèle sur les deux domaines **au sein d'un même lot**. L'idée est de conserver la diversité BF tout en imposant une priorité d'optimisation sur les exemples HF, considérés comme plus fiables.

Deux mécanismes sont appliqués conjointement :

1. **Co-training à ratio fixe par batch** : chaque lot contient exactement 16 images HF et 48 images BF (batch size = 64), soit un ratio HF de 25%.
2. **Loss pondérée par domaine** : la perte individuelle est multipliée par un poids plus fort sur HF

($w_{HF} = 2.0$) et plus faible sur BF ($w_{BF} = 0.7$), avant agrégation.

Configuration expérimentale utilisée :

Paramètre	Valeur	Commentaire
Architecture	ResNet-18	Entraînée <i>from scratch</i> , classifieur 10 classes
Optimiseur	Adam	Même famille d'optimisation que les expériences précédentes
Batch size	64	Composition contrôlée HF/BF à chaque itération
Époques	20	Entraînement conjoint sur 117 batches par époque
Ratio batch	16 HF / 48 BF	Ratio HF fixé à 25% par <i>MixedBatchSampler</i>
Poids de loss	HF : 2.0 ; BF : 0.7	Accent mis sur les erreurs HF dans la mise à jour des gradients
Données utilisées	1 881 HF train + 21 213 BF train	Validation sur 471 HF ; test sur 2 614 images

4.2.2 Résultats quantitatifs obtenus

Les résultats finaux de l'Étape 2 sont résumés ci-dessous pour les trois datasets (moyenne $\pm$ écart-type sur 3 graines). Le critère prioritaire reste la précision sur **Test HF**.

Dataset	Test HF	Test BF	Test Mixte	Coût donnée	Coût total
Animals-10	$63,8 \pm 5,2$	$61,0 \pm 4,2$	$62,4 \pm 4,7$	44 733	486 720
Imagewoof	$37,5 \pm 0,8$	$37,4 \pm 0,3$	$37,5 \pm 0,2$	17 116	183 040
Intel	$82,0 \pm 1,0$	$79,8 \pm 0,9$	$80,9 \pm 0,9$	26 652	291 200

Lecture rapide des courbes d'entraînement (Animals-10, graine 42)

La *train weighted loss* décroît régulièrement, avec une baisse plus rapide de la loss HF non pondérée que de la loss BF, ce qui confirme l'effet attendu du *reweighting*. La validation HF progresse globalement mais avec des oscillations intermédiaires marquées, signe d'une optimisation **plus instable** qu'en stratégie séquentielle — instabilité que confirme le fort écart-type inter-graines du

score final ($63,8 \pm 5,2\%$).

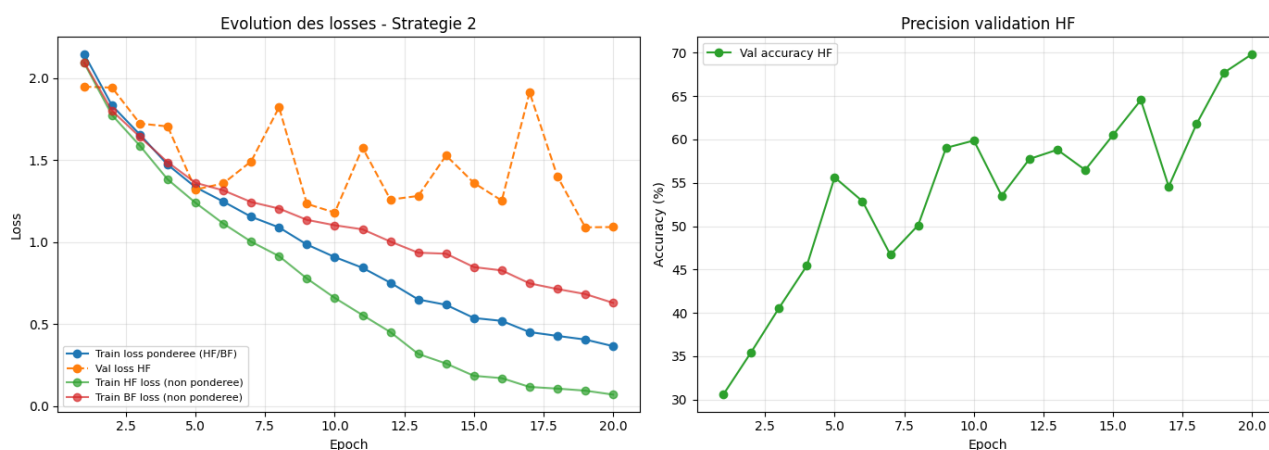


FIGURE 4.3 – Évolution des pertes et de la précision validation HF pour la Stratégie 2 (co-entraînement pondéré HF/BF) sur **Animals-10**.

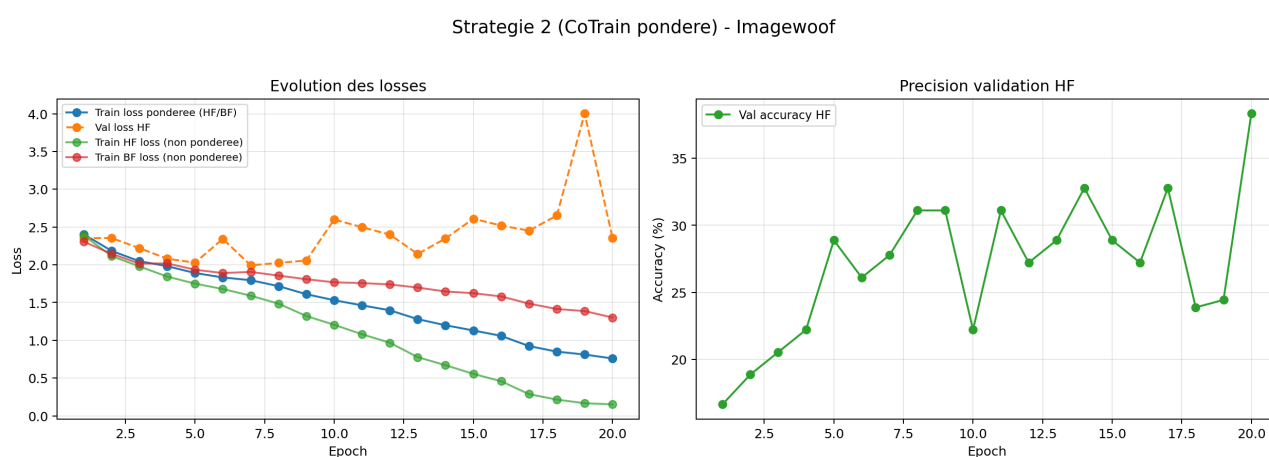


FIGURE 4.4 – Évolution des pertes et de la précision validation HF pour la Stratégie 2 sur **Imagewoof**.

4.2.3 Comparaison avec les baselines et la Stratégie 1

Modèle	HF	BF	Mixte	Coût donnée	Coût total
<i>Animals-10</i>					
BL 1 (HF)	54,3 ± 2,2	27,8 ± 0,8	41,1 ± 1,1	23 520	470 400
BL 2 (BF)	61,3 ± 6,3	71,0 ± 1,5	66,2 ± 2,7	21 213	424 260
BL 3 (Mixte)	74,0 ± 3,0	72,9 ± 1,1	73,4 ± 2,0	44 733	894 660
Stratégie 1	77,7 ± 0,6	68,3 ± 0,5	73,0 ± 0,3	44 733	400 230
Stratégie 2	63,8 ± 5,2	61,0 ± 4,2	62,4 ± 4,7	44 733	486 720
<i>Imagewoof</i>					
BL 1 (HF)	25,0 ± 4,1	22,1 ± 1,4	23,5 ± 2,6	8 990	179 800
BL 2 (BF)	42,7 ± 2,8	47,0 ± 1,6	44,9 ± 2,1	8 126	162 520
BL 3 (Mixte)	50,3 ± 2,5	48,8 ± 2,3	49,6 ± 2,4	17 116	342 320
Stratégie 1	51,0 ± 1,6	44,7 ± 1,3	47,9 ± 1,5	17 116	153 160
Stratégie 2	37,5 ± 0,8	37,4 ± 0,3	37,5 ± 0,2	17 116	183 040
<i>Intel</i>					
BL 1 (HF)	75,8 ± 2,2	47,0 ± 0,8	61,4 ± 1,1	14 020	280 400
BL 2 (BF)	78,7 ± 2,7	81,9 ± 1,4	80,3 ± 1,4	12 632	252 640
BL 3 (Mixte)	82,8 ± 1,2	82,1 ± 1,2	82,5 ± 1,2	26 652	533 040
Stratégie 1	86,1 ± 0,4	79,1 ± 1,4	82,6 ± 0,7	26 652	238 420
Stratégie 2	82,0 ± 1,0	79,8 ± 0,9	80,9 ± 0,9	26 652	291 200

Comparaisons clés (Animals-10) :

- **Gain HF modéré vs baselines** : la stratégie 2 atteint 63,8 % sur Test HF, devant BL 1 (54,3 %) et BL 2 (61,3 %), mais l'avantage sur BL 2 est faible (+2,5 points).
- **Dominée par la stratégie 1** : contre-intuitivement, la stratégie 2 est ici inférieure à la stratégie séquentielle **à la fois sur HF** (−13,9 points : 63,8 % vs 77,7 %) **et sur BF** (−7,3 points : 61,0 % vs 68,3 %). Voir les deux domaines en même temps ne suffit pas à égaler une spécialisation HF

finale explicite.

- **Instabilité marquée** : avec des écarts-types de 4 à 5 points, le co-training pondéré est nettement **moins stable** que les autres stratégies ($S1 < 1$ point), ce qui complique la sélection de modèle.
- **Coût intermédiaire** : son coût total (486 720 CA) est inférieur à BL 3 (894 660 CA) mais supérieur à la stratégie 1 (400 230 CA), pour une performance moindre.

Vérification sur Imagewoof et Intel

La Stratégie 2 se révèle **très sensible au volume HF et au type de tâche** :

- **Imagewoof (décrochage)** : 37,5 % HF seulement, sous BL 2 (42,7 %), BL 3 (50,3 %) et loin de la Stratégie 1 (51,0 %) ; seule BL 1 fait pire. Le faible volume HF (≈ 720 images train) ne permet pas au *reweighting* de spécialiser les *features* fine-grained, le batch étant mécaniquement dominé par les 48 échantillons BF sur 64. Son seul atout reste un équilibre HF/BF extrêmement resserré (37,5 / 37,4).
- **Intel (correcte mais jamais en tête)** : 82,0 % HF, au-dessus des baselines mono-domaine (BL 1, BL 2) mais en dessous de BL 3 (82,8 %) et surtout de la Stratégie 1 (86,1 %).

Bilan : sur les trois datasets, la Stratégie 2 n'est **jamais la meilleure** et souffre d'une forte variance ou d'un décrochage selon le dataset. Son intérêt conceptuel (équilibre HF/BF) ne se traduit pas en performance HF maximale.

4.2.4 Analyse : ce qui a marché, ce qui a moins marché, et enseignements

- **Équilibre HF/BF réel** : le mélange systématique HF/BF dans chaque batch produit l'écart HF–BF le plus resserré du panel (notamment sur Imagewoof), conforme à l'esprit de la méthode.
- **Effet du reweighting confirmé** : la décroissance plus rapide de la loss HF non pondérée montre que le modèle a effectivement priorisé les erreurs HF pendant l'optimisation.
- **Limite principale (objectif HF prioritaire)** : la stratégie 2 ne retrouve pas le niveau HF de la stratégie 1 — elle est même **dominée par S1 sur HF et BF** sur Animals-10 — ce qui confirme qu'une spécialisation finale explicite sur HF reste plus efficace pour viser la performance maximale sur domaine propre.
- **Instabilité et sensibilité au dataset** : forte variance inter-graines (jusqu'à ± 5 points) et décrochage sur Imagewoof (faible volume HF) ; la méthode est peu robuste sans réglage fin du ratio et des poids.

Conclusion Étape 2 (objectif HF prioritaire) : le co-training pondéré tient sa promesse d'équilibre HF/BF, mais il est **dominé par la stratégie 1** sur la métrique cible (Test HF) et souffre d'instabilité. Les stratégies suivantes (Curriculum, EWC) chercheront à retrouver — voire dépasser — le niveau HF de la stratégie 1 tout en améliorant la rétention BF et la stabilité.

4.2.5 Améliorations possibles de la méthode (Stratégie 2)

La stratégie de co-training pondéré offre une bonne robustesse inter-domaines ; son optimisation repose surtout sur le réglage fin du mélange de données et des poids de perte.

- **Ratio HF/BF dans le batch** : le choix 16/48 (25% HF) conditionne directement le compromis HF-BF. *Comment optimiser* : tester plusieurs ratios (20/80, 30/70, 40/60) et tracer la frontière de Pareto entre Test HF et Test BF.
- **Poids de loss w_{HF} et w_{BF}** : ce sont les hyperparamètres centraux de la méthode. *Comment optimiser* : recherche en grille ou bayésienne sur (w_{HF}, w_{BF}) (ex. $w_{HF} \in [1.0, 3.0]$, $w_{BF} \in [0.4, 1.0]$), en fixant d'abord le ratio batch.
- **Planning dynamique des poids** : garder des poids constants peut être sous-optimal selon la phase d'apprentissage. *Comment optimiser* : démarrer avec un poids HF modéré puis l'augmenter progressivement (curriculum de pondération).
- **Nombre d'époques** : des oscillations de validation suggèrent qu'un arrêt plus précoce peut être préférable à 20 époques fixes. *Comment optimiser* : early stopping sur validation HF avec patience et sauvegarde du meilleur modèle.
- **Learning rate et scheduler** : un LR fixe peut accentuer l'instabilité tardive. *Comment optimiser* : comparer LR fixes vs Cosine Annealing / OneCycleLR pour lisser la convergence.
- **Composition des batches (stochastique contrôlée)** : la variabilité intra-batch peut influencer la stabilité. *Comment optimiser* : conserver le ratio global mais imposer une diversité de classes par domaine (batch balancing par classe).
- **Fonction de loss alternative** : Balanced Softmax, focal reweighting, ou class-balanced loss peuvent mieux traiter classes rares et bruitées. *Comment optimiser* : protocole d'ablation en ne changeant qu'une composante de loss à la fois.
- **Qualité de la dégradation BF** : le niveau de bruit/sous-échantillonnage détermine l'écart de domaine. *Comment optimiser* : calibrer plusieurs intensités de dégradation puis choisir celle qui maximise la généralisation HF sans effondrer BF.
- **Régularisation et normalisation** : weight decay, EMA des poids, mixup/cutmix peuvent améliorer la robustesse de l'entraînement mixte. *Comment optimiser* : introduire les techniques progressivement et évaluer le gain marginal.
- **Validation multi-critères** : optimiser uniquement HF peut dégrader excessivement BF. *Comment optimiser* : utiliser un score composé (ex. $S = \alpha Acc_{HF} + (1 - \alpha) Acc_{BF}$) avec α élevé, cohérent avec la priorité HF.

4.3 Stratégie 3 – Curriculum Learning progressif

4.3.1 Protocole

Le *curriculum learning* (Bengio et al., 2009) consiste à présenter les exemples du plus simple au plus complexe, à l'image d'un apprentissage humain. Nous l'adaptions au cadre multi-fidélité en faisant croître **progressivement la proportion d'images HF** au fil d'un **entraînement unique et continu** : le réseau démarre sur le volume BF abondant et bon marché (structures grossières, faciles), puis la part de HF augmente par paliers jusqu'à un fine-tuning final entièrement HF. Contrairement à la Stratégie 1, il n'y a pas de rupture nette BF → HF mais une transition douce, ce qui vise à limiter l'oubli du domaine BF.

Le planning utilisé découpe les 20 époques en **quatre paliers** de 5 époques, à proportion d'images HF croissante :

Palier	Proportion HF	Rôle
Époques 1–5	0 %	Tout-BF : apprentissage des structures grossières sur le volume abondant
Époques 6–10	25 %	Introduction mesurée d'images nettes
Époques 11–15	60 %	Bascule majoritaire vers la haute fidélité
Époques 16–20	100 %	Fine-tuning final entièrement HF

Configuration : ResNet-18 *from scratch*, optimiseur Adam, *learning rate* 0,001, batch size 64, 20 époques, AMP (FP16) ; identique aux autres stratégies pour conserver la comparabilité.

4.3.2 Résultats quantitatifs obtenus

Résultats sur les trois datasets (moyenne $\pm$ écart-type sur 3 graines) ; critère prioritaire : **Test HF**.

Dataset	Test HF	Test BF	Test Mixte	Coût donnée	Coût total
Animals-10	76,5 $\pm$ 0,2	73,1 $\pm$ 0,1	74,8 $\pm$ 0,1	44 733	421 265
Imagewoof	47,5 $\pm$ 1,4	46,1 $\pm$ 1,1	46,8 $\pm$ 1,2	17 116	158 880
Intel	84,6 $\pm$ 1,0	82,6 $\pm$ 1,5	83,6 $\pm$ 1,2	26 652	250 880

Signature de la méthode

Le curriculum progressif se distingue par deux traits remarquables : la **meilleure rétention BF de toutes les stratégies** (sur Animals-10, 73,1 % en BF, soit même *au-dessus* de la baseline Mixte) et la **plus grande stabilité** inter-graines (écarts-types de l'ordre de 0,1–0,2 point sur Animals-10).

La transition douce BF → HF évite l'oubli brutal du domaine dégradé.

4.3.3 Comparaison avec les baselines et les stratégies précédentes

- **Vs baseline Mixte (BL 3)** : à coût donnée identique, le curriculum dépasse BL 3 sur Test HF des trois datasets (Animals-10 : 76,5 vs 74,0 ; Intel : 84,6 vs 82,8 ; Imagewoof : 47,5 vs 50,3 — ici légèrement en deçà), tout en réduisant fortement le coût total (Animals-10 : 421 265 vs 894 660 CA, soit −53%).
- **Vs Stratégie 1 (séquentielle)** : le curriculum est très proche en HF mais légèrement en retrait (Animals-10 : 76,5 vs 77,7 ; Intel : 84,6 vs 86,1), **en échange d'une bien meilleure rétention BF** (Animals-10 : 73,1 vs 68,3) et d'une stabilité supérieure. C'est le meilleur compromis HF/BF des stratégies orientées HF.
- **Vs Stratégie 2** : le curriculum domine nettement le co-training pondéré sur les trois datasets, en HF comme en stabilité.

4.3.4 Analyse : ce qui a marché, ce qui a moins marché, et enseignements

- **Ce qui a bien marché** : la montée progressive en HF combine les avantages des deux domaines — robustesse BF *conservée* et bonne spécialisation HF — avec une stabilité d'entraînement exemplaire.
- **Limite (objectif HF strict)** : pour la métrique cible unique (Test HF maximal), le curriculum reste un cran derrière les approches purement séquentielles (S1, et surtout EWC ci-après), la phase finale HF étant plus courte qu'un fine-tuning dédié complet.
- **Sensibilité fine-grained** : sur Imagewoof, le curriculum passe légèrement sous BL 3 en HF, signe que le faible volume HF limite l'efficacité de la phase finale.
- **Leçon principale** : si l'on cherche un modèle **équilibré et robuste** HF/BF à coût réduit, le curriculum progressif est l'option la plus sûre ; si seule la performance HF compte, une spécialisation finale plus marquée est préférable.

Conclusion Étape 3 : le curriculum learning progressif offre le **meilleur équilibre HF/BF** et la **meilleure stabilité** du panel, à coût donnée identique aux autres stratégies multi-fidélité. Il valide l'intérêt d'une transition douce entre domaines. L'Étape 4 (EWC) cherchera à pousser encore la performance HF tout en protégeant explicitement les acquis BF.

4.3.5 Améliorations possibles de la méthode (Stratégie 3)

- **Planning des paliers** : le découpage 0/25/60/100 % peut être affiné. *Comment optimiser* : rechercher le nombre de paliers, leur durée et les proportions HF (lissage continu plutôt que paliers, fonctions de montée linéaire/exponentielle).

- **Durée de la phase finale HF** : un dernier palier 100 % HF plus long rapprocherait le modèle d'un fine-tuning dédié. *Comment optimiser* : balayer la longueur du dernier palier en surveillant le compromis HF/BF.
- **Learning rate par palier** : un LR décroissant à chaque montée en HF stabiliserait la transition. *Comment optimiser* : associer un scheduler (Cosine, StepLR) au planning de curriculum.
- **Critère d'avancement adaptatif** : passer au palier suivant non pas à époque fixe mais selon la convergence. *Comment optimiser* : déclencher la montée en HF lorsque la validation plafonne (curriculum auto-pacé).
- **Régularisation** : weight decay / label smoothing pour réduire le sur-apprentissage de la phase HF finale. *Comment optimiser* : balayage court de ces hyperparamètres.

4.4 Stratégie 4 – Elastic Weight Consolidation (EWC)

4.4.1 Protocole

L'*Elastic Weight Consolidation* (Kirkpatrick et al., 2017) a été conçu pour lutter contre l'**oubli catastrophique** en apprentissage continu. Nous l'appliquons ici comme une amélioration directe de la Stratégie 1 : le schéma reste **séquentiel BF → HF**, mais pendant le fine-tuning HF, une pénalité empêche le modèle de trop s'éloigner des poids appris sur BF, donc de **conserver sa robustesse BF** tout en se spécialisant sur HF.

Concrètement, après le pré-entraînement BF (poids de référence θ^*), on estime l'**importance** de chaque poids pour la tâche BF via la diagonale de la **matrice d'information de Fisher** F (calculée sur 1 000 échantillons BF). Le fine-tuning HF minimise alors :

$$\mathcal{L}_{tot} = \mathcal{L}_{HF} + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^*)^2,$$

où λ contrôle la force du rappel vers les poids « importants pour BF ». Plus F_i est grand, plus le poids θ_i est ancré.

Paramètre	Valeur	Commentaire
Architecture	ResNet-18	<i>From scratch</i> , tête 10 classes (6 pour Intel)
Optimiseur	Adam	Identique aux autres stratégies
Époques BF / HF	10 / 10	Pré-entraînement BF puis fine-tuning HF (comme S1)
Learning rates	0,001 puis 0,0003	Plus faible en fine-tuning
Force EWC (λ)	400	Intensité du rappel vers les poids BF
Échantillons Fisher	1 000	Images BF servant à estimer l'importance des poids

4.4.2 Résultats quantitatifs obtenus

Résultats sur les trois datasets (moyenne $\pm$ écart-type sur 3 graines) ; critère prioritaire : **Test HF**.

Dataset	Test HF	Test BF	Test Mixte	Coût donnée	Coût total
Animals-10	78,0 $\pm$ 0,3	68,8 $\pm$ 0,7	73,4 $\pm$ 0,4	44 733	400 230
Imagewoof	53,3 $\pm$ 0,7	46,8 $\pm$ 0,6	50,0 $\pm$ 0,6	17 116	153 160
Intel	85,8 $\pm$ 0,4	80,0 $\pm$ 0,9	82,9 $\pm$ 0,6	26 652	238 420

Effet de la pénalité de Fisher

Par rapport à la Stratégie 1 (même pipeline séquentiel, même coût), EWC améliore **à la fois** le Test HF **et** la rétention BF : sur Animals-10, +0,3 point HF (78,0 vs 77,7) et +0,5 point BF (68,8 vs 68,3) ; sur Imagewoof, +2,3 points HF (53,3 vs 51,0) et +2,1 points BF (46,8 vs 44,7). La pénalité de Fisher remplit donc son rôle : se spécialiser sur HF *sans* sacrifier BF.

4.4.3 Comparaison globale des stratégies

Le tableau de synthèse ci-dessous rassemble **toutes les approches** (3 baselines + 4 stratégies) sur les **trois datasets**. Pour chaque dataset, le meilleur score Test HF est mis en évidence.

Modèle	HF	BF	Mixte	Coût donnée	Coût total
<i>Animals-10</i>					
BL 1 (HF)	54,3 ± 2,2	27,8 ± 0,8	41,1 ± 1,1	23 520	470 400
BL 2 (BF)	61,3 ± 6,3	71,0 ± 1,5	66,2 ± 2,7	21 213	424 260
BL 3 (Mixte)	74,0 ± 3,0	72,9 ± 1,1	73,4 ± 2,0	44 733	894 660
Stratégie 1	77,7 ± 0,6	68,3 ± 0,5	73,0 ± 0,3	44 733	400 230
Stratégie 2	63,8 ± 5,2	61,0 ± 4,2	62,4 ± 4,7	44 733	486 720
Stratégie 3	76,5 ± 0,2	73,1 ± 0,1	74,8 ± 0,1	44 733	421 265
Stratégie 4 (EWC)	78,0 ± 0,3	68,8 ± 0,7	73,4 ± 0,4	44 733	400 230
<i>Imagewoof</i>					
BL 1 (HF)	25,0 ± 4,1	22,1 ± 1,4	23,5 ± 2,6	8 990	179 800
BL 2 (BF)	42,7 ± 2,8	47,0 ± 1,6	44,9 ± 2,1	8 126	162 520
BL 3 (Mixte)	50,3 ± 2,5	48,8 ± 2,3	49,6 ± 2,4	17 116	342 320
Stratégie 1	51,0 ± 1,6	44,7 ± 1,3	47,9 ± 1,5	17 116	153 160
Stratégie 2	37,5 ± 0,8	37,4 ± 0,3	37,5 ± 0,2	17 116	183 040
Stratégie 3	47,5 ± 1,4	46,1 ± 1,1	46,8 ± 1,2	17 116	158 880
Stratégie 4 (EWC)	53,3 ± 0,7	46,8 ± 0,6	50,0 ± 0,6	17 116	153 160
<i>Intel</i>					
BL 1 (HF)	75,8 ± 2,2	47,0 ± 0,8	61,4 ± 1,1	14 020	280 400
BL 2 (BF)	78,7 ± 2,7	81,9 ± 1,4	80,3 ± 1,4	12 632	252 640
BL 3 (Mixte)	82,8 ± 1,2	82,1 ± 1,2	82,5 ± 1,2	26 652	533 040
Stratégie 1	86,1 ± 0,4	79,1 ± 1,4	82,6 ± 0,7	26 652	238 420
Stratégie 2	82,0 ± 1,0	79,8 ± 0,9	80,9 ± 0,9	26 652	291 200
Stratégie 3	84,6 ± 1,0	82,6 ± 1,5	83,6 ± 1,2	26 652	250 880
Stratégie 4 (EWC)	85,8 ± 0,4	80,0 ± 0,9	82,9 ± 0,6	26 652	238 420

Lectures clés du tableau de synthèse :

- **EWC, meilleure stratégie pour l'objectif HF** : EWC obtient le meilleur Test HF sur Animals-10 (78,0 %) et Imagewoof (53,3 %), et talonne la Stratégie 1 sur Intel (85,8 vs 86,1 %) — le tout au **coût le plus bas** (identique à S1) et avec une excellente stabilité.
- **EWC domine la Stratégie 1** : à pipeline et coût identiques, EWC est supérieur ou égal à S1 en HF *et* en BF sur les trois datasets : la pénalité de Fisher est un ajout « gratuit » bénéfique.
- **Curriculum, meilleur équilibre** : la Stratégie 3 offre la meilleure rétention BF et la meilleure stabilité, au prix d'un HF légèrement inférieur à EWC/S1.
- **Toutes les stratégies orientées HF battent les baselines à coût réduit** : S1, S3 et S4 dépassent la borne supérieure BL 3 sur Test HF (Animals-10 et Intel) tout en divisant le coût total par environ deux ; seule la Stratégie 2 reste en retrait.

4.4.4 Analyse : ce qui a marché, ce qui a moins marché, et enseignements

- **Ce qui a bien marché** : la pénalité de Fisher atteint son objectif — spécialiser le modèle sur HF *sans* effondrer la robustesse BF — ce qui fait d'EWC la meilleure approche pour la cible HF, à coût minimal.
- **Gain le plus net sur le fine-grained** : c'est sur Imagewoof (faible volume HF) que l'apport d'EWC sur la Stratégie 1 est le plus marqué (+2,3 points HF), là où la protection des acquis BF compte le plus.
- **Limite** : EWC ajoute deux hyperparamètres (λ , nombre d'échantillons Fisher) et un coût de calcul de la matrice de Fisher ; le réglage de λ conditionne l'équilibre spécialisation/rétention.
- **Leçon principale** : ancrer explicitement les poids importants pour le domaine bon marché est un levier simple et efficace pour pousser la performance HF sans payer la robustesse BF.

Conclusion Étape 4 — bilan des stratégies : pour l'objectif HF prioritaire à coût réduit, le classement qui se dégage est **EWC** $\gtrsim$ **Stratégie 1** > **Curriculum** > **Co-training**, toutes nettement meilleures que les baselines à coût total équivalent ou inférieur. **EWC** est recommandé lorsque la performance HF prime ; le **Curriculum** lorsqu'un équilibre HF/BF robuste est recherché. La phase suivante (optimisation Optuna) cherchera à raffiner les hyperparamètres de chacune de ces stratégies.

4.4.5 Améliorations possibles de la méthode (Stratégie 4)

- **Force du rappel (λ)** : hyperparamètre central. *Comment optimiser* : balayer λ (ex. [10, 1000] en échelle log) et tracer le compromis HF/BF résultant.
- **Estimation de Fisher** : 1 000 échantillons est un compromis. *Comment optimiser* : faire varier le nombre d'échantillons et comparer Fisher diagonale vs approximations plus riches.

- **EWC en ligne / multi-tâches** : accumuler l'importance sur plusieurs domaines (BF de différentes intensités). *Comment optimiser* : EWC en ligne (online EWC) avec mise à jour incrémentale de F .
- **Couplage avec le curriculum** : appliquer EWC *pendant* une montée progressive en HF. *Comment optimiser* : combiner planning de curriculum et pénalité de Fisher, puis mesurer le gain conjoint.
- **Learning rate de fine-tuning** : interagit fortement avec λ . *Comment optimiser* : recherche conjointe (λ, lr_{HF}) plutôt que séparée.

ewpage

Bibliographie

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. *Deep Residual Learning for Image Recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [2] Diederik P. Kingma, Jimmy Ba. *Adam : A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR), 2015.
- [3] Ian Goodfellow, Yoshua Bengio, Aaron Courville. *Deep Learning*. MIT Press, 2016. Disponible en ligne : <https://www.deeplearningbook.org>
- [4] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, Li Fei-Fei. *Image-Net Large Scale Visual Recognition Challenge*. International Journal of Computer Vision (IJCV), 123(3) :211–252, 2015.
- [5] Dan Hendrycks, Thomas Dietterich. *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations*. International Conference on Learning Representations (ICLR), 2019.
- [6] Marc Peter Deisenroth, Jun Wei Ng. *Distributed Gaussian Processes*. International Conference on Machine Learning (ICML), 2015.
- [7] Marc C. Kennedy, Anthony O'Hagan. *Predicting the Output from a Complex Computer Code When Fast Approximations Are Available*. Biometrika, 87(1) :1–13, 2000.
- [8] Mengye Ren, Wenyuan Zeng, Bin Yang, Raquel Urtasun. *Learning to Reweight Examples for Robust Deep Learning*. International Conference on Machine Learning (ICML), 2018.
- [9] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, Masashi Sugiyama. *Co-teaching : Robust Training of Deep Neural Networks with Extremely Noisy Labels*. Advances in Neural Information Processing Systems (NeurIPS), 2018.
- [10] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, Jason Weston. *Curriculum Learning*. Proceedings of the 26th International Conference on Machine Learning (ICML), 2009.
- [11] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, et al. *Overcoming Catastrophic Forgetting in Neural Networks*. Proceedings of the National Academy of Sciences (PNAS), 114(13) :3521–3526, 2017.
- [12] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, Masanori Koyama. *Optuna : A Next-generation Hyperparameter Optimization Framework*. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), 2019.